ROTEIRO DE LABORATÓRIO PRÁTICO 1

Fundamentos de Grafos e Matriz de Adjacência em C

Disciplina: Matemática Discreta
Curso: Bacharelado em Sistemas de Informação
Instituição: IFES - Campus Colatina

1. Introdução Conceitual Teórica

No estudo da Teoria dos Grafos, modelamos estruturas relacionais complexas por meio de nós (ou vértices) e conexões (arestas ou arcos).

- **Grafo Não Orientado ($G = (V, E)$):** É composto por um conjunto de vértices V e um conjunto de arestas E que estabelecem conexões **bidirecionais** (não ordenadas) entre os pares de vértices.
- **Grafo Direcionado (Dígrafo):** As arestas possuem uma orientação específica (setas), indicando um sentido de percurso de um vértice de origem para um vértice de destino.
- **Grau de um Vértice:** Representa o número de arestas incidentes a um vértice. Em *dígrafos*, o conceito desdobra-se em **grau de emissão** (quantidade de arcos que saem do vértice) e **grau de recepção** (quantidade de arcos que chegam ao vértice).
- **Casos Especiais:**
  - *Laço:* Aresta que conecta um vértice a ele mesmo.
  - *Vértice Isolado / Sumidouro:* Vértice que não possui conexões ou que possui grau zero em determinadas direções.
- **Modelagem Computacional (Matriz de Adjacência):** Para representar um grafo com V vértices em linguagem C, utilizamos uma matriz estática bidimensional `int adj[V][V]`. Adotamos a convenção binária: 1 indica a existência de uma aresta e 0 a sua ausência. Nos grafos não direcionados, a matriz possui a propriedade de simetria ($A = A^T$).

2. Descrição das Imagens para Referência Visual

Referência de Diagramas Teóricos:
• **Grafo Não Orientado vs. Simples:** Visualiza estruturas conexas, ausência de arestas e grafos simples/não simples com laços.
• **Dígrafo e Matriz de Adjacência:** Associa visualmente os arcos direcionados e o mapeamento de bits na matriz estática $V \times V$.

3. Conjunto de Exercícios de Programação em C

Abaixo estão os 4 exercícios práticos para implementação em linguagem C:

EXERCÍCIO 1.1 Inicialização e Inserção de Arestas

Inicializar estaticamente uma matriz de adjacência $N \times N$ com zeros e implementar uma função `void adicionar_aresta(int u, int v, int direcionado)` que marque a conexão correspondente (tratando a simetria caso o parâmetro `direcionado` seja falso).

EXERCÍCIO 1.2 Cálculo do Grau de um Vértice

Criar uma função `int grau_vertice(int v, int n)` que calcule e retorne o grau de um dado vértice somando os valores de sua linha correspondente na matriz de adjacência.

EXERCÍCIO 1.3 Graus de Emissão e Recepção em Dígrafos

Implementar as funções `int grau_emissao(int v, int n)` e `int grau_recepcao(int v, int n)` para dígrafos, verificando por meio da matriz se determinado vértice atua como um sumidouro ou elemento isolado.

EXERCÍCIO 1.4 Verificação de Grafos Regulares

Escrever uma função `int eh_regular(int n)` que percorra todos os vértices da matriz e verifique se todos possuem exatamente o mesmo grau, retornando 1 se verdadeiro ou 0 caso contrário.